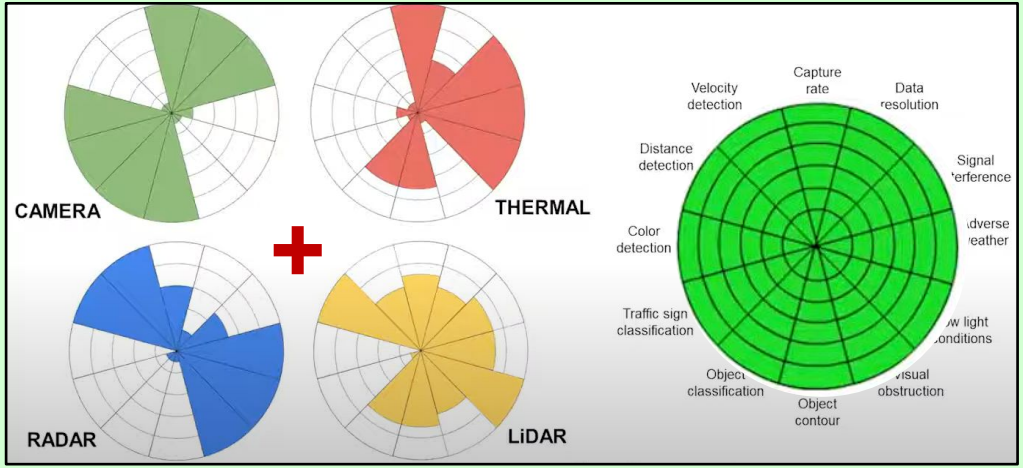
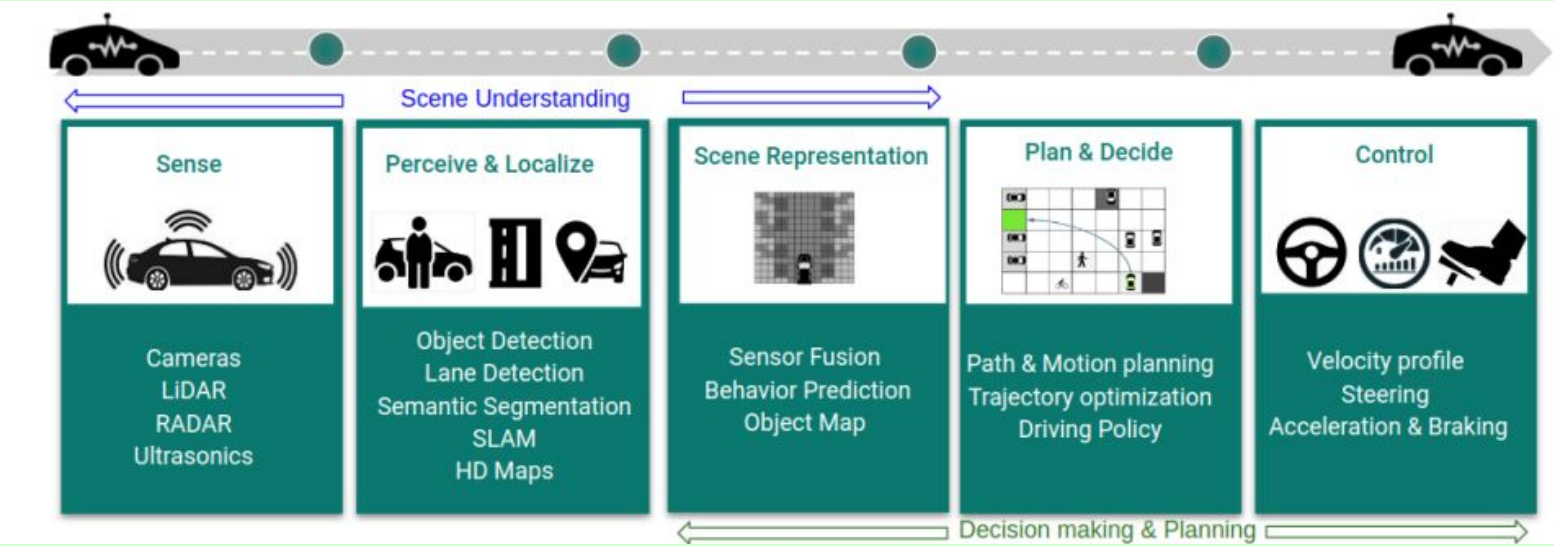
Deep Reinforcement Learning with Optimal Sensor Selection and Fusion for Autonomous Driving

Tushar Mehta,
Parkland High School,
Grade 9



Background

- Promise of Autonomous Vehicles (AV) for reducing traffic accidents, congestion and carbon footprint is dependent on the safety of these systems in all conditions.
- Reinforcement learning based on sensor fusion is a novel idea for object detection and path planning.
- This project is aimed at testing different sensing methodologies to achieve better driving performance, safety and avoid collisions.



Definitions

- **Deep Reinforcement Learning (DRL)**: A generative approach which interactively learns from the environment based on rewards linked to different control actions.
- **Ego Vehicle**: AV of primary interest - Evaluating performance in diverse scenarios.
- **Multiple Object Tracking (MOT):** DRL algorithm that initializes, confirms, predicts, corrects, and deletes the tracks of moving objects based on sensors
- **Most Important Object (MIO):** is the closest track in the front in the ego lane

Sensor	Primary Purpose	Advantage	Drawback
RGB Camera	Object and Lane Detection	Accurate and Low Cost	Only Works in good light
RaDAR: Radio Detection And Ranging	Distance, Direction, Velocity	All-Weather; Day/Night	Low Resolution, Classification
LiDAR: Light Detection And Ranging	3D information, Distance	High Accuracy, All-weather	Expensive, Rain/Fog

Research Study

Goal: Develop an efficient senor fusion technique based on RGB cameras and a Radar to be used in a machine learning algorithm for improved driving performance, safety and collision avoidance.

- Implement Autonomous Emergency Braking (AEB) to help mitigate collisions.
- Create a highway lane following algorithm with vision processing, sensor fusion, and controller components

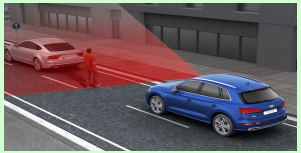
Research Questions:

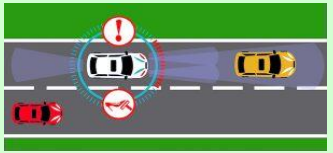
- Which sensors and their perception parameters are effective in tracking objects?
- Is there a closed-loop method for combining multi modal sensor outputs to achieve safe and optimal driving path for autonomous driving?

Hypothesis and Design Space

Autonomous vehicles with a closed-loop machine learning model built on sensor fusion data for the improved perception of the environment leads to better driving metrics. Radar with longer range sensing together with RGB camera will lead to more robust tracker of the objects for collision avoidance.

Applications: **Autonomous Emergency Braking (AEB)** **Highway Lane Following (HLF)**





Independent Variables:

1. Radar parameters (FoV, Resolution & Range)
2. Camera parameters (Focal length)
3. Driving scenarios (Pedestrian crossing, Cut-in)

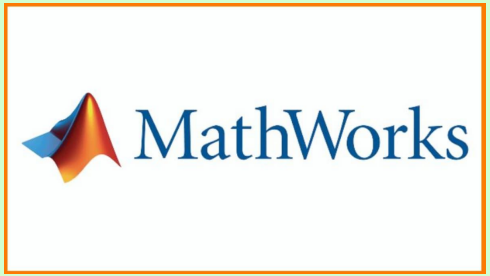
Dependent Variables:

1. Lateral Lane Deviations (Y_1)
2. Collision Mitigation (Y_2)
3. Headway: MIO Distance (Y_3)

Controlled Variables: Vehicle, sensor and environment models

AV Simulators and Model Libraries

- Software/Modeling Environment:
  - MATLAB: Automated Driving, Sensor Fusion & Controls toolboxes, Simulink
  - CARLA simulator: Python IDE jupyter notebook, Pandas, Matplotlib, sklearn
  - Unreal engine for driving scenarios and simulations
- Sensor Data: Radar point clouds & 360^0 RGB data from MATLAB models
- MATLAB model libraries, driving scenarios for training
- GPU Computing Environment



Procedure

A. Scenarios/Sensors Selection	B. Model Development & Simulations	C. Performance Assessment
• Five driving scenarios • Radar + five different cameras for 360^0 view • Pick AV simulation engine • Select the models for the vehicle dynamics, sensors & control logic in MATLAB	• Implement & test algorithms for sensor fusion and MOT. • Perform parametric study to understand the relative importance of radar and camera parameters for different driving scenarios.	• Compute and compare the driving metrics for MOT and AEB models • Apply the best models for MOT and collision avoidance on the Highway lane following case study .

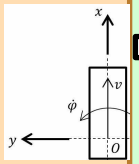
Closed-Loop Sensing and Driving Algorithm

Scene Selection

✓ Environment ✓ Road Network ✓ Actors(**x, v**): Pedestrians, other vehicles

Ego Vehicle Dynamics Model

- Throttle
- Steering angle



Sensor Selection/Configuration

RGB Camera

Focal length (
Optical center
Image size pr

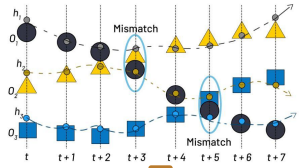
Radar

[AzFoV, 5]
RadarRange

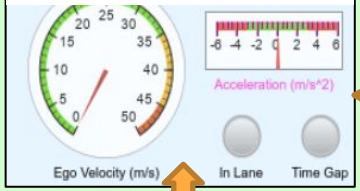
IMU + GNSS
Ego Location & Velocity

Sensor Fusion

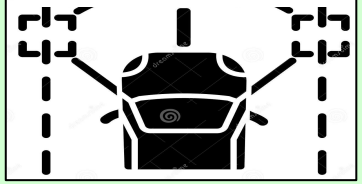
Multi Object Tracker



Metrics Assessment



Lane Following

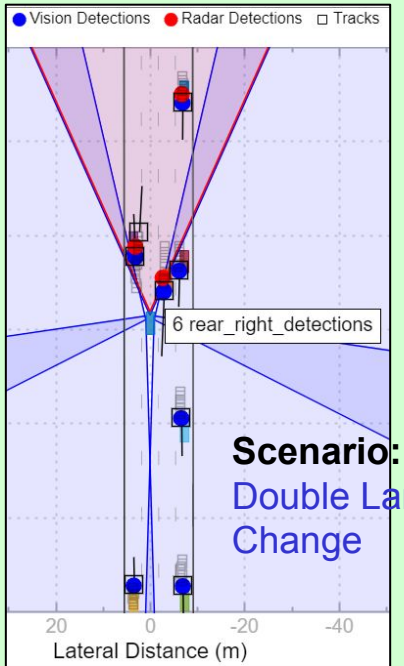


AEB

Headway Controller

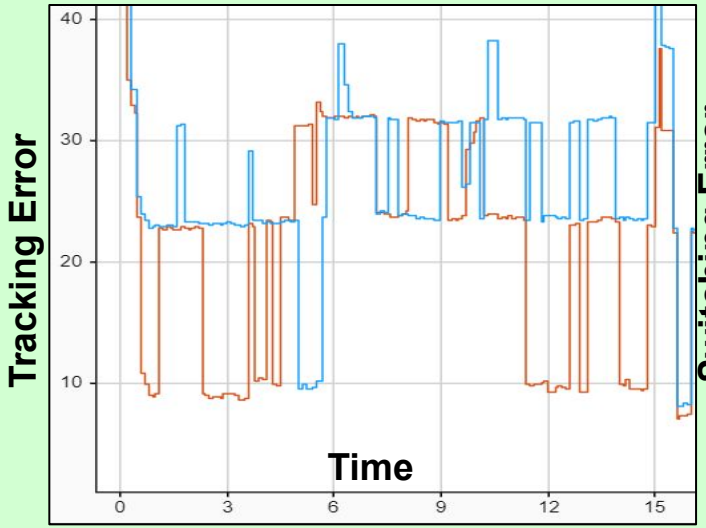
Multi Object Tracker: Fusion of 1 Radar + 5 Cameras

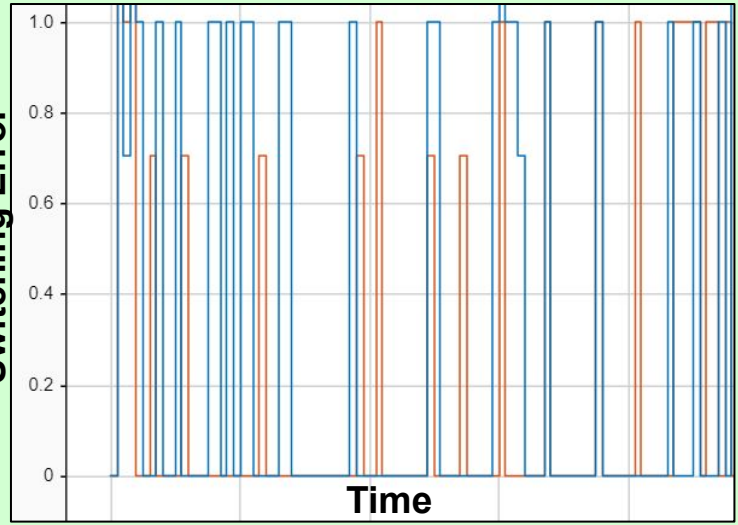
- Radar detections are clustered and fused with vision detections based on DRL to develop a joint probabilistic tracking model for multiple objects.
- Metrics: (1) **Tracking Error** – Error between a set of tracks and their ground truths. (2) **Switching Error** – Indicates the resulting error during track switching



Sensor Configuration 1: Radar FOV: 48, Camera focal len. (Front: 920, Rear: 600)

Sensor Configuration 2: Radar FOV: 90, Camera focal len. (Front: 800, Rear: 500)

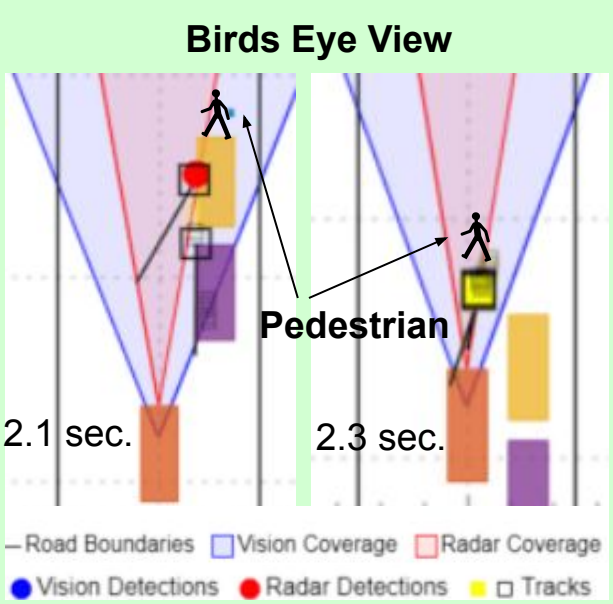


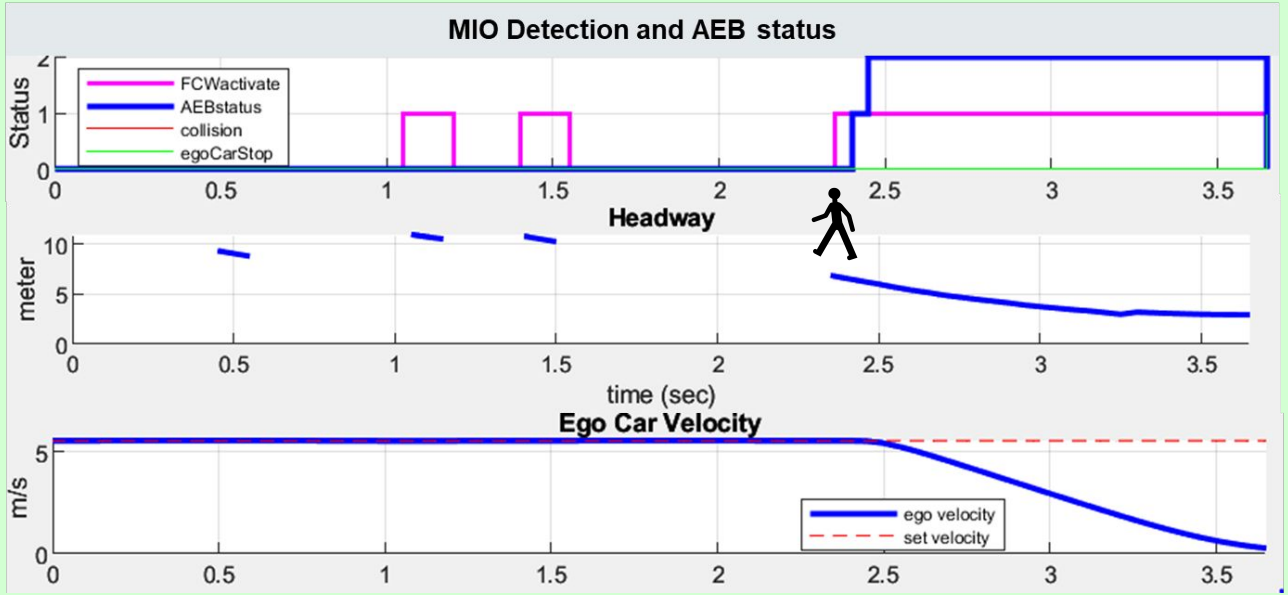


AEB: Model and Control

Scenario: Pedestrian crossing on the near side. Initially, blocked by the other vehicles.

1. Tracker indicates a new MIO at around 2.3 s. Headway drops suddenly.
2. Forward collision activated
3. AEB activated at 2.4 sec. Ego vehicle decelerates
4. Vehicle comes to stop at 3.7 sec, within 4m of the pedestrian. ***Collision Avoided!***



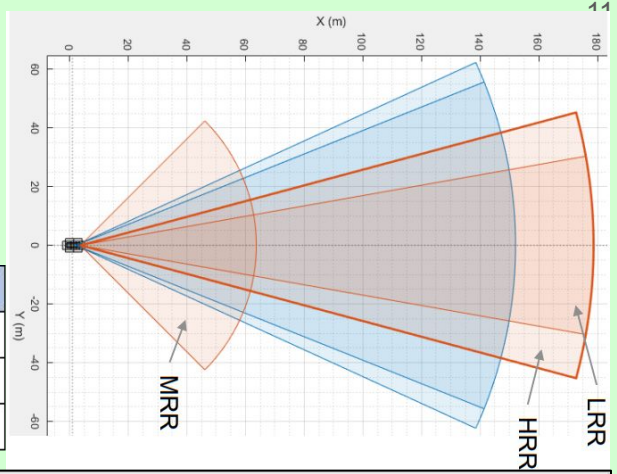


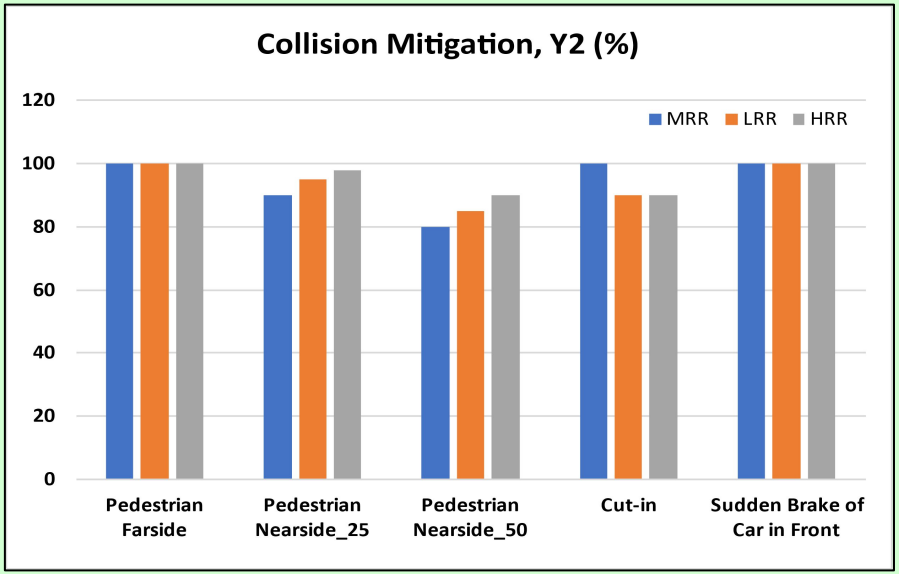
AEB Performance Sensitivities

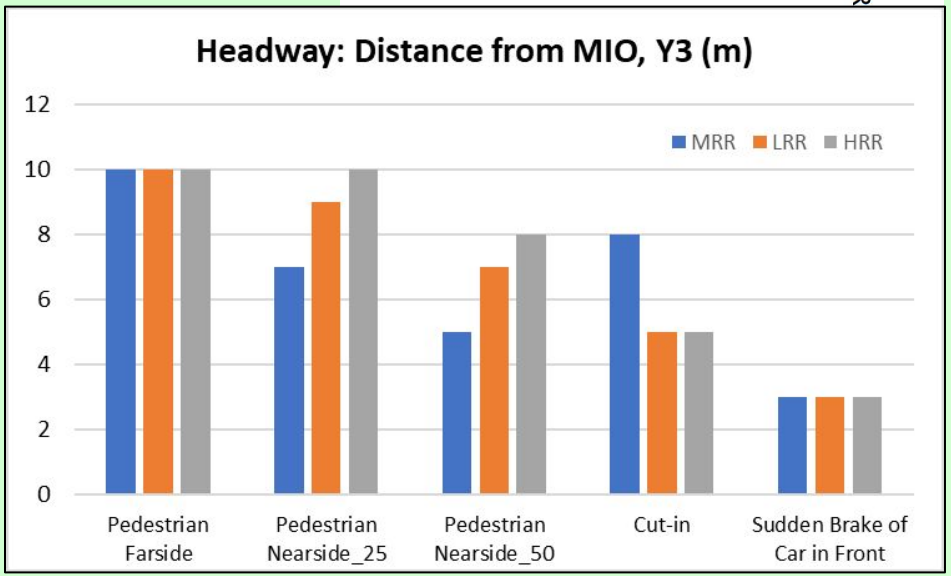
- Driving Performance was tested with 3 different configurations of Radar- Mid-range (MRR), Long-range(LRR) & High-Resolution (HRR)
  ★ **HRR performed Best except in the "Cut-in" scenario**

- RGB camera tested at two different focal lengths: **No Significant effect.**

	MRR	LRR	HRR
Azimuthal Field of View(deg)	90	20	30
Range Resolution(m)	1.25	2.5	1.25
Azimutal Resolution(deg)	12	3.5	2



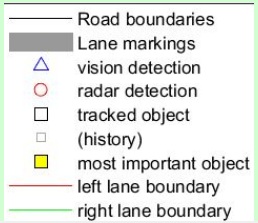


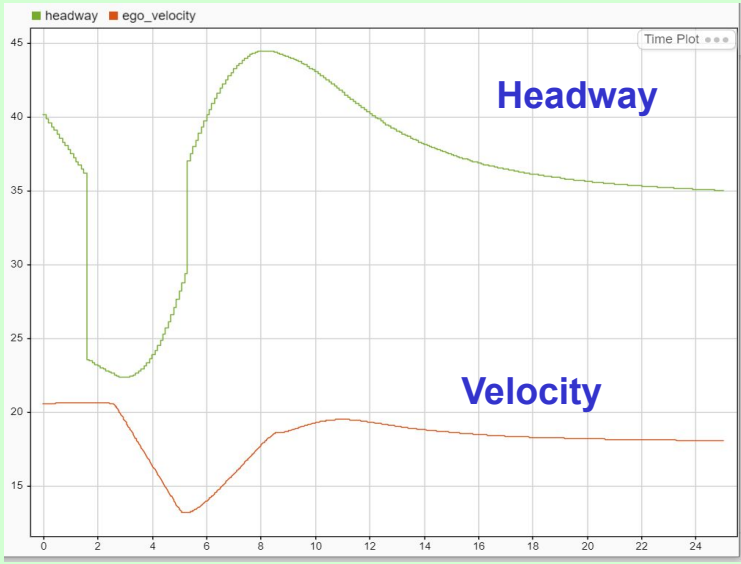


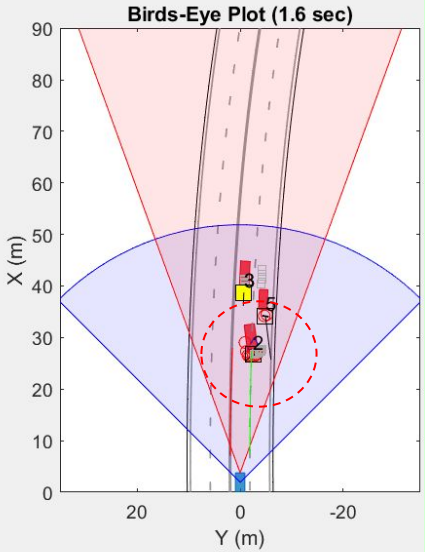
Highway Lane Following Validation Case Study

Scenario: Close cut-in & cut-out on 2-lane curved highway. Multiple cars; varying speeds

Success: Follow the lane (minimal lateral deviations, Y_1) without collisions (safe Headway, Y_2)







Code (m and slx files) Screenshots

Scenario and Vehicle parameter setup

```
%% Scenario parameters
assignin('base','scenario',scenario);
assignin('base','assessment',assessment);
assignin('base', 'egoActorID', scenario.Actors(1).ActorID);

%% Vehicle parameters
egoVehDyn = egoVehicleDynamicsParams(scenario);
assignin('base','egoVehDyn', egoVehDyn);
assignin('base','vehSim3D',  vehicleSim3DParams(scenario));
assignin('base','actorProfiles', actorProfiles(scenario));

%% Sensor parameters
egoVehicle = scenario.Actors(1);
assignin('base','camera',    cameraParams(egoVehicle));
assignin('base','radar',     radarParams(egoVehicle));

%% General model parameters
assignin('base','Ts',0.1);                    % Algorithm sample time
assignin('base','v_set', egoVehDyn.VLong0);   % Set velocity (m/s)

%% Path following controller parameters
assignin('base','tau',             0.5);      % Time constant for lo
assignin('base','time_gap',        1.5);      % Time gap
assignin('base','default_spacing', 10);       % Default spacing
assignin('base','max_ac',          2);        % Maximum acceleration
```

Base sensor models

Sensor Selection and Configuration

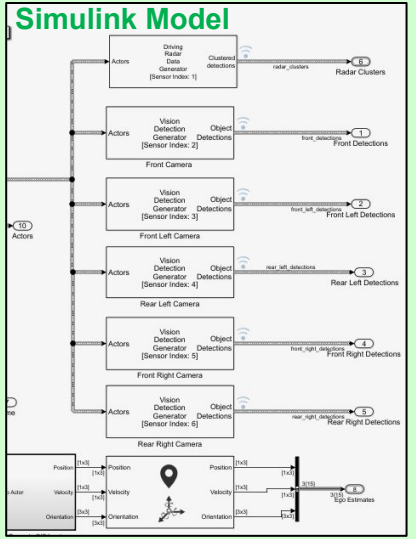
```
function radar = radarParams(egoVehicle)
% Radar sensor parameters
radar.FieldOfView     = [40,5];   % Field of view (degrees)
radar.DetectionRanges = [1,150];  % Ranges (m)
radar.Position        = ...       % Position with respect to
    [ egoVehicle.Length - egoVehicle.RearOverhang, ...
      0,...
      0.8];
radar.PositionSim3d   = ...       % Position with respect to
    radar.Position - ...
    [ egoVehicle.Length/2 - egoVehicle.RearOverhang, 0, 0];
radar.Rotation = [ 0, 0, 0];      % [roll, pitch, yaw] (deg)
end
```

```
function camera = cameraParams(egoVehicle)
% Camera sensor parameters
camera.NumColumns       = 1024;    % Number of columns in
camera.NumRows          = 768;     % Number of rows in cam
camera.FieldOfView      = [45,45]; % Field of view (degree
camera.ImageSize        = [camera.NumRows, camera.NumColum
camera.PrincipalPoint   = [camera.NumColumns,camera.NumRow
camera.FocalLength      = [camera.NumColumns / (2*tand(cam
                           camera.NumColumns / (2*tand(cam
camera.Position         = ...      % Position with respect
    [ 1.8750, ...                  %  - X (by the rear-vie
      0,...                        %  - Y (center of vehic
      1.2];                        %  - Height
```

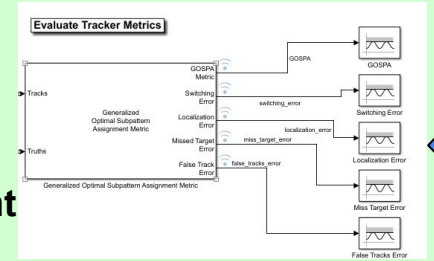
Sensor Fusion

```
%% Tracking and sensor fusion parameters
assignin('base','M',            3);       % Tracker M value for M-out-of-N logic  (N/A)
assignin('base','N',            4);       % Tracker N value for M-out-of-N logic  (N/A)
assignin('base','P',            5);       % Tracker P value for deletion logic  (N/A)
assignin('base','Q',            5);       % Tracker Q value for deletion logic  (N/A)
assignin('base','trackRegisterThreshold',0.5); % Threshold for track register
assignin('base','trackAssignmentThreshold',200);% Threshold for track assignment
assignin('base','numTracks',    100);     % Maximum number of tracks                (N/A)
assignin('base','numSensors',  6);        % Maximum number of sensors      (N/A)

%% GOSPA parameters
assignin('base','cutoffDist',  30);       % Cutoff distnace
assignin('base','orderOfGOSPA', 2);       % order of GOSPA metric
assignin('base','alpha',    2);           % Alpha parameter of GOSPA
assignin('base','switchingPenalty',1);    % Penalty for assignment switching

%% General model parameters
assignin('base','Ts',0.1);
```

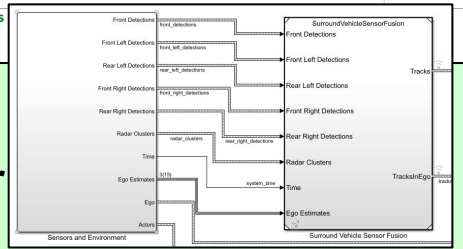
Simulink Model



Metrics Assessment



MOT Tracker



Conclusions

- Sensor fusion based on radar point cloud and 360^0 RGB vision data improve the perception of the environment needed for closed loop Autonomous driving.
- High resolution radar together with RGB cameras perform well as a robust Multi-object Tracker of the objects for collision avoidance in the AEB application
- MOT tracking and switching errors are minimized with increase in radar FOV and smaller focal lengths of the front and rear cameras.
- Radar resolution parameters are more influential than the field of view or camera focal length to achieve 90+% levels for collision mitigation.
- The closed loop DRL-based sensor fusion algorithm was successfully validated on the highway lane following application under close cut-in/cut-out scenario.

Future Research

- **LIDAR**: Incorporate and evaluate LIDAR based Reinforcement Learning in the closed loop decision-making.
- **Field Testing**: Implement and evaluate the algorithms in the AV test kits.
- **CARLA-MATLAB Integration** for testing more complex real-world scenarios

Applications

- **Autonomous Driving Safety**: Closed-loop sensor fusion is the key for full adoption of autonomous vehicles.
- **Industrial Safety**: Multi-modal sensing deployed on robots/drones is beneficial for achieving next level of safety in chemical plants & factories.
- **Human Assistant**: Sensor fusion has a huge potential to be the eyes and ears for people suffering from blindness and deafness.

References

1. Carrasco, Serge-Paul. Machine Learning Models for Autonomous Vehicles, 22 Aug. 2020. https://asiliconvalleyinsider.com/2020/08/22/modeling-for-automous-vehicles/
2. Dosovitskiy, A., Ros, G., Codevilla, F., Lopez, A. & Koltun, V.. CARLA: An Open Urban Driving Simulator, Proceedings of the 1st Annual Conference on Robot Learning in Proceedings of Machine Learning Research, 2017, 78:1-16.
3. Els, Peter. "Intelligent sensor fusion for smart cars." 2017. https://www.automotive-iq.com/autonomous-drive/articles/intelligent-sensor-fusion-smart-cars
4. Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, Andreas Geiger. TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving, 2022, arXiv:2205.15997.
5. Laganiere, Robert. Sensor fusion for autonomous vehicles: Strategies methods and tradeoffs Synopsis Conference, 2021, https://www.youtube.com/watch?v=2Fcmh7SLPBI
6. The Math Works, Inc. MATLAB. Version 2022b, The Math Works, Inc., 2022. Computer Software. www.mathworks.com/.